

รายงานการทำงาน: ระบบสกิล (Skill System)

โปรเจกต์ acter of souls | branch Unity_26/08/26 | 29 สิงหาคม 2026

งานรอบนี้คือทำให้ระบบสกิลที่เขียนค้างไว้ รันได้จริง เพื่อเริ่มเทสต์ โดยกดปุ่ม E กับ Q แล้วสกิลออก มีคูลดาวน์ และดาเมจเข้าสู่ตราเดียวกับการโจมตีปกติ

1. สิ่งที่ทำ

แก้ไฟล์เดิม 2 ไฟล์ เพิ่มไฟล์ใหม่ 1 ไฟล์ และจัดข้อมูลสกิลอีก 2 asset

ไฟล์	สถานะ	ทำอะไร
Assets/Scripts/Skills/SkillCaster.cs	ใหม่	ตัวยิงสกิล อ่านปุ่ม E / Q เก็บคูลดาวน์ แล้วเรียก Cast()
Assets/Scripts/Skills/ActiveSkillData.cs	แก้	เพิ่มธาตุของสกิล และทำ Melee / AOE / Buff ให้ทำงานจริง
Assets/Scripts/Characters/Player/PlayerAttack.cs	แก้	เปิดให้สกิลใช้สูตรดาเมจเดิมได้ โดยส่งธาตุกับดาเมจของสกิลเข้าไป
Assets/Data/Skill/_MegaBlast.asset	แก้	ค่าเดิมเป็นค่าทดสอบที่ใช้ไม่ได้แล้ว ปรับให้อยู่ในสเกลปกติ
Assets/Data/Skill/_SlashWave.asset	ใหม่	สกิลแบบ Melee ไว้ใช้ช่อง Q สำหรับเทสต์

ผลที่ได้

- กด E ยิงสกิลระยะไกล กด Q พันระยะประชิดพร้อมปลั๊กศัตรู ทั้งคู่มีคูลดาวน์
- สกิลทั้ง 4 แบบ (Melee, Ranged, AOE, Buff) ทำงานได้จริง จากเดิมที่ทำงานแค่ Ranged แบบเดียว
- ดาเมจสกิลได้ครีต ได้หักเกราะศัตรู ได้ใส่ damage popup และติดสถานะตอนโดน เหมือนการโจมตีปกติ
- ดาเมจของสกิลบวกเพิ่มจากพลังโจมตีผู้เล่น และธาตุยึดตามสกิล ไม่ใช่ตามอาวุธที่ถือ

2. ปัญหาเดิมก่อนแก้

ระบบสกิลมีไฟล์ครบ แต่ไม่เคยถูกรันจริงสักครั้ง เพราะขาดชิ้นส่วนที่เชื่อมทุกอย่างเข้าด้วยกัน

- ไม่มีใครเรียก Cast() กันทั้งโปรเจกต์แล้วไม่เจอจุดที่เรียกเลย ระบบสกิลจึงเป็นโค้ดที่ไม่เคยทำงาน
- ไม่มีที่เก็บคูลดาวน์ ตัวแปร cooldown มีในข้อมูลสกิล แต่ไม่มีใครนับถอยหลังให้
- สกิล 3 ใน 4 แบบเป็นของว่าง Melee กับ AOE เป็นแค่ Debug.Log ส่วน Buff เป็น case ว่างเปล่า
- ดาเมจสกิลไม่ผ่านสูตรกลาง ยังเข้า TakeDamage ตรง ๆ เลยไม่มีครีต ไม่หักเกราะศัตรู ไม่มีสีตามธาตุ และไม่ติดสถานะตอนโดน
- ช่อง E / Q บน HUD มีแต่ภาพ ใน Persistence.prefab มี UI/HUD/Skill พร้อมป้ายปุ่ม แต่ไม่มีสคริปต์ผูกอยู่เลย

3. แนวคิดในการออกแบบ

3.1 คูลดาวน์ต้องเก็บไว้นอกไฟล์สกิล

ActiveSkillData เป็น ScriptableObject ซึ่งก็คือไฟล์เดียวที่ทุกตัวละครใช้ร่วมกัน ถ้าเก็บตัวจบเวลาไว้ในนั้น พอผู้เล่นร้ายสกิล ศัตรูที่ใช้สกิลเดียวกันจะติดคูลดาวน์ตามไปด้วย เพราะมันคือตัวแปรก่อนเดียวกัน จึงย้ายคูลดาวน์ไปเก็บที่ SkillCaster ซึ่งติดอยู่กับตัวละครแต่ละตัว

3.2 ใช้สูตรดาเมจเดิม ไม่เขียนสูตรใหม่

ถ้าเขียนสูตรดาเมจของสกิลแยกอีกชุด เวลาปรับสมดุลเกมจะต้องไล่แก้สองที่ และมีโอกาสสั้ม จึงเลือกเดิมพารามิเตอร์ 2 ตัวเข้าไปในเมธอดเดิม คือ เราดูที่จะใช้แทนของอาวุธ และดาเมจที่จะบวกเพิ่ม ทั้งสองตัวมีค่าเริ่มต้น (default) ทำให้จุดที่เรียกอยู่เดิมทั้งหมดไม่ต้องแก้แม้แต่จุดเดียว

3.3 อ่านปุ่มจากคีย์บอร์ดตรง ๆ ไม่แตะไฟล์ inputactions

ในไฟล์ Player.inputactions ไม่มีปุ่ม Q กับ E อยู่เลย ส่วนใน UI.inputactions มีอยู่แล้ว แต่ผูกเป็นปุ่มสลับหน้าเมนู ถ้าไปเพิ่มหรือแก้ไฟล์พวกนี้เสี่ยงกระทบระบบเมนูที่ใช้งานได้อยู่ จึงอ่านจาก Keyboard.current ตรง ๆ แบบเดียวกับปุ่มทดสอบของ StatusEffectManager ที่โปรเจกต์ทำไว้อยู่แล้ว แล้วเพิ่มเงื่อนไขว่าถ้าเมนูเปิดอยู่ให้ข้ามไปเพื่อไม่ให้กด Q ตอนเปิดเมนูแล้วสกิลออกพร้อมกับสลับหน้า

3.4 ใช้ของที่มีอยู่แล้ว ไม่สร้างใหม่

ของเดิมที่เอามาใช้	ใช้ทำอะไร
PlayerAttack.enemyLayers	เลเยอร์ศัตรูของ Melee และ AOE ไม่ต้องเพิ่มช่องใหม่ในสกิล
PlayerAttack.DamageCalculation	สูตรดาเมจทั้งหมด คริต เกราะ popup ติดสถานะ
playerControl.aimDirection	ทิศเล็งจากเมาส์ ส่งเข้า Cast()
ObjectPooling.SpawnFromPool	สร้างกระสุนสกิล สร้าง pool ให้เองถ้ายังไม่มี
StatusEffectController.ApplyEffect	ทางเข้าของสกิลแบบ Buff ใช้ระบบสถานะเดิมทั้งก้อน
PlayerAttack.CanAct	สถานะอย่าง Paralyzed สั่งห้ามร้ายสกิลได้ฟรี ๆ

4. โค้ดเก่าเทียบโค้ดใหม่

4.1 PlayerAttack.cs – สูตรดาเมจ

ของเดิมบังคับว่าเราต้องมาจากอาวุธเสมอ และไม่มีที่ให้บวกดาเมจของสกิลเข้าไป

โค้ดเก่า

```
public void DamageCalculation(EnemyHealth targetEnemy, float multiple, weaponsData attackWeapon)
{
    ElementType attackElement = attackWeapon != null ? attackWeapon.elementType : ElementType.None;

    float weaponMultiplier = GetWeaponMultiplier(attackWeapon);

    float totalAttack = playerStats.baseAttack + GetWeaponBaseAttack(attackWeapon) * weaponMultiplier;
```

ของใหม่เพิ่มพารามิเตอร์ 2 ตัวที่มีค่าเริ่มต้น ส่วนสูตรข้างในไม่แตะเลย

โค้ดใหม่

```
// overrideElement = ระบุของสกิล (None = ใช้ระบุของอาวุธตามเดิม)
// bonusDamage      = ดาเมจของตัวสกิลเอง บวกเข้ากับ baseAttack ของผู้เล่น
public void DamageCalculation(EnemyHealth targetEnemy, float multiple, weaponsData attackWeapon,
                             ElementType overrideElement = ElementType.None, float bonusDamage = 0f)
{
    ElementType attackElement = overrideElement != ElementType.None
        ? overrideElement
        : (attackWeapon != null ? attackWeapon.elementType : ElementType.None);

    float weaponMultiplier = GetWeaponMultiplier(attackWeapon);

    float totalAttack = playerStats.baseAttack + GetWeaponBaseAttack(attackWeapon) * weaponMultiplier + bonusDamage;
```

แล้วเพิ่มทางเข้าสำหรับสกิลโดยเฉพาะ ของเดิมไม่มีเมธอดนี้

โค้ดใหม่ (เพิ่มเข้ามา)

```
// สกิลไม่ได้มาจากการกดโจมตี จึงไม่มี currentAttackWeapon ของตัวเอง
// ยืมอาวุธที่สวมอยู่มาคิดดาเมจฐาน/คริต แต่ระบุใช้ของสกิล
public void ApplySkillDamage(EnemyHealth targetEnemy, ActiveSkillData skill)
{
    if (targetEnemy == null || skill == null) return;

    weaponsData weapon = null;
    if (slot1 != null && !slot1.IsEmpty) weapon = slot1.GetCurrentItem() as weaponsData;
    else if (slot2 != null && !slot2.IsEmpty) weapon = slot2.GetCurrentItem() as weaponsData;

    DamageCalculation(targetEnemy, skill.damageMultiplier, weapon, skill.elementType, skill.baseDamage);
}
```

4.2 ActiveSkillData.cs — สกิลแบบ Melee / AOE / Buff

ของเดิม Melee กับ AOE พิมพ์ข้อความออกมาเฉย ๆ ส่วน Buff เป็น case ว่าง กดแล้วไม่มีอะไรเกิดขึ้น

โค้ดเก่า

```
case ActiveSkillType.Buff:
    // ใส่บัฟให้ตัวละคร
    break;

private void PerformMeleeAttack(GameObject caster, Vector2 aimDirection)
{
    Debug.Log($"Cast Melee Skill: {skillName}");
}

private void PerformAOEAttack(GameObject caster)
{
    Debug.Log($"Cast AOE Skill: {skillName}");
}
```

ของใหม่ Melee กวาดด้วยกล่อง AOE กวาดด้วยวงกลม สองแบบนี้ต่างกันแค่รูปทรง ส่วนที่เหลือจึงแยกออกมาใช้ร่วมกันใน DamageInside

โค้ดใหม่

```
// ล้อ PlayerAttack.TriggerSpearDamage - กล้องวางล่องหน้าตามทิศเล็ง และเอียงตามมุมเล็ง
private void PerformMeleeAttack(GameObject caster, PlayerAttack attacker, Vector2 aimDirection)
{
    if (attacker == null) return;

    float aimAngle = Mathf.Atan2(aimDirection.y, aimDirection.x) * Mathf.Rad2Deg;
    Vector2 center = (Vector2)caster.transform.position + aimDirection.normalized * meleeStats.meleeRange;

    Collider2D[] hits = Physics2D.OverlapBoxAll(center, meleeStats.hitBoxSize, aimAngle, attacker.enemyLayers);
    DamageInside(hits, attacker, aimDirection);
}

private void PerformAOEAttack(GameObject caster, PlayerAttack attacker)
{
    if (attacker == null) return;

    Collider2D[] hits = Physics2D.OverlapCircleAll(caster.transform.position, effectRadius, attacker.enemyLayers);
    DamageInside(hits, attacker, Vector2.zero);
}

// Melee กับ AOE ต่างกันแค่รูปทรงก็กวาด ส่วนที่เหลือใช้ร่วมกัน
// knockbackDir = Vector2.zero แปลว่าไม่ผลัก (AOE ไม่มีทิศทาง)
private void DamageInside(Collider2D[] hits, PlayerAttack attacker, Vector2 knockbackDir)
{
    // ศัตรูตัวเดียวอาจมีหลาย collider - กันไม่ให้กันดาเมจซ้ำในครั้งเดียว
    HashSet<EnemyHealth> hitAlready = new HashSet<EnemyHealth>();

    foreach (Collider2D hit in hits)
    {
        EnemyHealth enemyHealth = hit.GetComponentInParent<EnemyHealth>();
        if (enemyHealth == null || !hitAlready.Add(enemyHealth)) continue;

        attacker.ApplySkillDamage(enemyHealth, this);
        ApplyKnockback(hit, knockbackDir);
        SpawnHitEffect(hit.transform.position);
    }
}

// บัพใช้ระบบสถานะตัวเดิมทั้งหมด - ระยะเวลา สดึก ไอคอน VFX มาจาก StatusEffectSO ไม่ทำซ้ำที่นี่
private void PerformBuff(GameObject caster)
{
    if (buffEffect == null) return;

    IStatusEffectReceiver receiver = caster.GetComponent<IStatusEffectReceiver>();
    receiver?.ApplyEffect(buffEffect);
}
```

4.3 ActiveSkillData.cs — ตอนกระสุนโดนศัตรู

ของเดิมคิดดาเมจเองสองบรรทัด ไม่ผ่านสูตรกลาง และใช้ GetComponent ซึ่งหาไม่เจอ ถ้าตัว collider เป็นลูกของศัตรูอีกที

โค้ดเก่า

```
private void OnProjectileHit(GameObject caster, Collider2D hitCollider)
{
    Debug.Log($"{skillName} hit {hitCollider.name}!");

    var enemyHealth = hitCollider.GetComponent<EnemyHealth>();
    if (enemyHealth != null)
    {
        float finalDamage = baseDamage * damageMultiplier;
        enemyHealth.TakeDamage(finalDamage);
    }
    ...
}
```

โค้ดใหม่

```
private void OnProjectileHit(PlayerAttack attacker, Collider2D hitCollider)
{
    // เดิมใช้ GetComponent ทำให้พลาดเมื่อ collider เป็นลูกของตัวศัตรู - ใช้ InParent เหมือนฝั่ง Melee
    EnemyHealth enemyHealth = hitCollider.GetComponentInParent<EnemyHealth>();
    if (enemyHealth != null && attacker != null)
    {
        attacker.ApplySkillDamage(enemyHealth, this);
    }

    SpawnHitEffect(hitCollider.transform.position);
}
```

4.4 SkillCaster.cs — ไฟล์ใหม่ทั้งไฟล์

ของเดิมไม่มีไฟล์นี้ นี่คือชิ้นส่วนที่หายไปและทำให้ระบบสกิลไม่เคยทำงาน ส่วนที่ยกมาข้างล่างคือหัวใจของมัน

โค้ดใหม่

```
// ช่องสกิล 1 ช่องบน HUD: ปุ่มที่กด + สกิลที่ใช้ได้
[System.Serializable]
public class SkillSlot
{
    public Key key = Key.E;
    public ActiveSkillData skill;

    // นับถอยหลังของช่องนี้โดยเฉพาะ ไม่ใช้ไว้ใน Inspector เพราะเป็นค่า runtime
    [HideInInspector] public float cooldownTimer;
}

// ปุ่ม Q/E ในเกมเพลย์ไม่มีอยู่ใน Player.inputactions จึงอ่านจาก Keyboard ตรง ๆ
private void ReadInput()
{
    if (Keyboard.current == null) return;    // กันเครื่องที่ไม่มีคีย์บอร์ด

    // ตอนเปิดเมนู Q/E คือปุ่ม Previous/Next ของ UI ห้ามให้ร่ายสกิลซ้อน
    if (UiPanelController.instance != null && UiPanelController.instance.IsPanelOpen) return;

    foreach (SkillSlot slot in slots)
    {
        if (Keyboard.current[slot.key].wasPressedThisFrame)
            TryCast(slot);
    }
}

public void TryCast(SkillSlot slot)
{
    if (slot.skill == null) { Debug.Log($"[Skill] ช่อง {slot.key} ว่าง"); return; }

    if (slot.cooldownTimer > 0f)
    {
        Debug.Log($"[Skill] {slot.skill.skillName} ยังติดคูลดาวน์อีก {slot.cooldownTimer:F1} 5");
        return;
    }

    // เชื่อมโยงเดียวกับที่ PlayerAttack.Update ใช้กับการโจมตีซ้อน บวกด้านสถานะ (Paralyzed ปิด CanAct)
    if (attackScript != null && (attackScript.isAttacking || attackScript.isShooting || !attackScript.CanAct))
    {
        Debug.Log($"[Skill] {slot.skill.skillName} ร่ายไม่ได้ตอนนี้");
        return;
    }

    Vector2 aimDirection = playerController != null ? playerController.aimDirection : Vector2.right;
    slot.skill.Cast(gameObject, aimDirection);

    slot.cooldownTimer = slot.skill.cooldown;
    Debug.Log($"[Skill] ร่าย {slot.skill.skillName} (ปุ่ม {slot.key})");
}
```

5. ตารางเทียบก่อน-หลัง

หัวข้อ	ก่อนแก้	หลังแก้
การร่ายสกิล	ไม่มีใครเรียก Cast() เลย	กด E / Q แล้วร่ายผ่าน SkillCaster
คูลดาวน	มีตัวเลขในข้อมูล แต่ไม่มีใครนับ	SkillCaster นับถอยหลังแยกทีละช่อง
สกิล Melee	Debug.Log เฉย ๆ	กวาดกลองตามทิศเล็ง เข้าดาเมจ และปลั๊กศัตรู
สกิล AOE	Debug.Log เฉย ๆ	กวาดวงกลมรอบตัว เข้าดาเมจทุกตัวในรัศมี
สกิล Buff	case ว่าง ไม่ทำอะไร	ติดสถานะให้ตัวเองผ่านระบบสถานะเดิม
สูตรดาเมจ	baseDamage x damageMultiplier แล้วยังเข้า TakeDamage ตรง ๆ	เข้าสู่ตรงกลางเต็มรูปแบบ ได้ครีต เกราะศัตรู popup สี และติดสถานะตอนโดน
ธาตุของสกิล	ไม่มี ใช้ธาตุของอาวุธก็ต่อ	สกิลมีธาตุของตัวเอง และชนธาตุอาวุธ
ดาเมจของสกิล	เป็นดาเมจดิบ ไม่เกี่ยวกับผู้เล่น	บวกเข้ากับ baseAttack ของผู้เล่น + ดาเมจอาวุธ
ตอนกระสุนโดน	GetComponent หา EnemyHealth พลาดถ้า collider เป็นลูก	GetComponentInParent เหมือนฝั่ง Melee
กันดาเมจซ้ำ	ไม่มี	HashSet กันศัตรูตัวเดียวกันดาเมจซ้ำในครั้งเดียว
ชนกับปุ่มอื่น	-	เช็คว่ามีปุ่มเปิดอยู่ไหมก่อนร่าย ปุ่ม 1-4 ไม่กระทบ

การเปลี่ยนค่าข้อมูลสกิล

ค่าใน _MegaBlast.asset เดิมเป็นค่าที่ตั้งไว้ลองเล่น พอดาเมจเข้าสู่ตรงจริงแล้วค่าพวกนี้ใช้ไม่ได้

ช่อง	ค่าเดิม	ค่าใหม่	เหตุผล
damageMultiplier	2.1312311e+10	1.5	ตอนนี้เป็นตัวคูณสุดท้าย ตำแหน่งเดียวกับ 0.25 ของหอก ไม่ใช่ดาเมจดิบ
baseDamage	99999	40	กลายเป็นดาเมจที่บวกเข้ากับพลังโจมตีผู้เล่น ต้องอยู่สเกลเดียวกับอาวุธ
elementType	()	Fire	ช่องใหม่ ถ้าไม่ตั้งจะเป็น None แล้วไม่ติดสถานะ popup ไม่มีสี
cooldown	0	2	คูลดาวน 0 ทำให้ทดสอบระบบคูลดาวนไม่ได้

6. จุดดี จุดเสีย

จุดดี

- ระบบสกิลรันได้จริงครั้งแรก เอาไปทดสอบได้เลย ไม่ต้องรอส่วนอื่น
- สูตรดาเมจยังมีที่เดียว ไม่ได้แตกเป็นสูตรของอาวุธกับสูตรของสกิล เวลาปรับสมดุลแก้ทีเดียวจบ และสกิลได้ของแถมทั้งหมดก็การโจมตีปกติมีอยู่แล้ว
- ความเสี่ยงพังต่ำ พารามิเตอร์ที่เพิ่มมีค่าเริ่มต้น จุดที่เรียกเมรอดเดิมอยู่ 2 จุด จึงไม่ต้องแก้เลย พฤติกรรมการโจมตีปกติเหมือนเดิมทุกอย่าง
- ข้อมูลเก่าไม่เสีย ในไฟล์สกิลมีแต่การเพิ่มช่องใหม่ ไม่มีการลบหรือเปลี่ยนชื่อช่อง asset เดิมกับ GUID จึงอยู่ครบ
- ปรับได้จาก Inspector จะเปลี่ยนปุ่ม สลับสกิล หรือเพิ่มช่องที่ 3 ทำได้โดยไม่ต้องแก้โค้ด
- ไม่ชนกับของเดิม ปุ่ม 1-4 ยังเป็นปุ่มทดสอบสถานะเหมือนเดิม และไม่ได้แตะไฟล์ inputactions ที่ระบบเมนูใช้อยู่
- ใช้ของเดิมซ้ำเยอะ เลเยอร์ศัตรู ระบบ pool ระบบสถานะ ทิศเล็ง ใช้ตัวที่มีอยู่แล้วทั้งหมด ไม่ได้สร้างของซ้ำซ้อน

จุดเสีย

- ยังไม่ได้คอมไพล์ ผมเปิด Unity Editor จากตรงนี้ไม่ได้ ต้องให้ Unity build ก่อนถึงจะยืนยันได้ว่าไม่มี error
- ยังต้องลากใส่ Inspector เอง ต้อง Add Component และลากสกิลใส่ช่องด้วยมือ ยังไม่ได้ผูกกับอาวุธอัตโนมัติ ทั้งที่ weaponsData มีช่อง first / second / third เตรียมไว้แล้ว
- HUD ยังไม่แสดงอะไร ช่อง E / Q ยังเป็นกล่องขาว ไม่มีไอคอนสกิลและไม่มีวงแหวนคูลดาวนั้ ต้องดูคูลดาวนั้จาก Console เอา
- อ่านปุ่มตรง ๆ ไม่ผ่าน Input System แลความปลอดภัยมาด้วยข้อเสียคือ ผู้เล่นเปลี่ยนปุ่มเองไม่ได้ และจอย gamepad ใช้ไม่ได้ ถ้าจะทำจริงต้องย้ายไปเป็น action ที่หลัง
- ยืมอาวุธช่องแรกมาคิดดาเมจ ถ้าถืออาวุธ 2 เล่ม ดาเมจสกิลจะอิงเล่มซ้ายเสมอ ตอนนี้อยอมรับได้เพราะสกิลยังไม่ผูกกับอาวุธ แต่พอผูกแล้วต้องแก้ไขเล่มที่เป็นเจ้าของสกิล
- ไม่มีอนิเมชันตอนร้าย ช่อง castAnim ในข้อมูลสกิลยังไม่มีใครอ่าน สกิลจึงออกทันทีที่กดปุ่ม ต่างจากการฟันหอกที่รอจังหวะจาก Animation Event
- castEffectPrefab ยังรวบอยู่ ตอนร้ายมีการ Instantiate เอฟเฟกต์แล้วไม่เคย Destroy ถ้าใส่ prefab ช่องนี้แล้วร้ายรัว ๆ เอฟเฟกต์จะสะสมในตัวผู้เล่นเรื่อย ๆ เป็นปัญหาเดิมที่ยังไม่ได้แก้ เพราะตัดออกตอนย่อขอบเขตงาน
- Melee เช็การชนแค่เฟรมเดียว ศัตรูที่วิ่งเข้ามาหลังจากเฟรมนั้นจะไม่โดน ซึ่งเป็นพฤติกรรมเดียวกับการฟันหอกอยู่แล้ว แต่ควรรู้ไว้

7. สิ่งที่ต้องทำต่อใน Unity

โค้ดเสร็จแล้ว เหลืองานผูกใน Editor ซึ่งทำจากตรงนี้ได้

ขั้น	ทำอะไร
1	เปิด Assets/prefab/Core/Persistence.prefab
2	หา GameObject ที่มี playerControl กับ PlayerAttack แล้วกด Add Component เลือก SkillCaster
3	ลาก playerController และ attackScript ใส่ช่อง (อยู่บน GameObject เดียวกัน)
4	ตั้ง Slots เป็น Size 2 แล้วกรอก ช่อง [0] ปุ่ม E สกิล _MegaBlast / ช่อง [1] ปุ่ม Q สกิล _SlashWave

วิธีเช็คว่าใช้ได้

ทดสอบ	สิ่งที่ควรเห็น
กด E เล็งใส่ศัตรู	กระสุนออก มี log [Skill] ร่าย MegaBlast ตามด้วย Total Damage dealt:
กด E รัว ๆ	ขึ้น log ว่ายังติดคุลดาวนั แล้วร่ายได้อีกครั้งหลัง 2 วินาที
ย่นติดศัตรูแล้วกด Q	ศัตรูเสียเลือดและถูกสลักถอย ตัวเดียวเสียเลือดครั้งเดียว
ตั้ง baseDamage จาก 40 เป็น 0	ตัวเลข Total Damage dealt ลดลงชัดเจน แปลว่าดาเมจสกิลถูกบวกจริง
ถืออาวุธธาตุ Fire แล้วกด Q (Wind)	สี damage popup เป็นสีของ Wind ไม่ใช่ Fire
กด Tab เปิดเมนูแล้วกด Q หรือ E	สลับหน้าเมนูอย่างเดียวนะ ไม่มี log [Skill]
กด 1 2 3 4	ติดสถานะเหมือนเดิม ไม่มีสกิลออก
ลบสกิลออกจากช่อง Q แล้วกด Q	ขึ้น log ว่าช่องว่าง ไม่มี error

งานที่เหลือไว้ทำต่อ

- ใส่ไอคอนและวงแหวนคุลดาวนับนช่อง E / Q — โครง Slot กับ Fill ที่มีอยู่ตรงกับ StatusEffectIconSlot (background กับ fill) พอดี เอามาใช้ซ้ำได้เลย แต่ต้องใส่รูป icon ให้สกิลก่อน เพราะตอนนี้ยังว่าง
- ผูกสกิลกับอาวุธ — weaponsData มีช่อง first / second / third ประกาศไว้แล้วแต่ยังไม่มีใครอ่าน
- _PowerUp.asset เสีย — ถูกบันทึกไว้กับโครงสร้างเก่าของ PassiveSkillData ทำให้ GetBonusPercent คืน 0 เสมอ
- UI สกิลรั้งยังไม่ได้ผูก — ช่อง Passive / Active / grid / ui ใน SkillTreeManager ยังว่างทั้งหมด
- ข้อผิดพลาดใน HUD — GameObject ชื่อ mouseLeft ในช่องสกิลที่สอง จริง ๆ แสดงรูปปุ่ม Q ควรเปลี่ยนชื่อ

8. สรุป

ก่อนหน้านี้ระบบสกิลมีไฟล์ครบแต่ไม่เคยทำงาน เพราะไม่มีใครเรียก Cast() และไม่มีที่เก็บคุลดาวนั

งานรอบนี้เติมชิ้นส่วนที่ขาดเข้าไปคือ SkillCaster ทำให้กด E กับ Q แล้วสกิลออกจริง

นอกจากนั้นยังเติมสกิลอีก 3 แบบที่เป็นของว่างให้ทำงาน และย้ายดาเมจสกิลมาใช้สูตรกลางเดียวกับการโจมตีปกติ โดยเพิ่มพารามิเตอร์ 2 ตัวที่มีค่าเริ่มต้น จึงไม่กระทบจุดที่เรียกอยู่เดิม สกิลเลยได้ทั้งครีต การหักเกราะศัตรู สี damage

popup และการติดสถานะ พร้อมกับมีดาเมจและธาตุเป็นของตัวเองตามที่ต้องการ

แนวทางหลักคือใช้ของที่มีอยู่แล้วให้มากที่สุด กังเลเยอร์ศัตรู ระบบ pool ระบบสถานะ และทิศเล็ง
และไม่แตะของที่ใช้งานได้อยู่ คือไม่แก้ไฟล์ inputactions ไม่แก้จุดที่เรียกสูตรดาเมจเดิม
และไม่ลบช่องข้อมูลเก่าในไฟล์สกิล

ที่ยังขาดคือส่วนแสดงผลบน HUD การผูกสกิลกับอาวุธ และการย้ายปุ่มไปอยู่ใน Input System รวมถึงต้องให้ Unity
คอมไพล์ยืนยันก่อน เพราะยังไม่ได้ build

ไฟล์ที่เกี่ยวข้อง: SkillCaster.cs (ใหม่), ActiveSkillData.cs, PlayerAttack.cs, _MegaBlast.asset, _SlashWave.asset (ใหม่)